

T. VIKAS

2303A52281

1480. Running Sum of 1d Array

```
class Solution(object):  
    def runningSum(self, nums):  
        running_sum = 0  
        ans = []  
        for i in range(0, len(nums)):  
            running_sum += nums[i]  
            ans.append(running_sum)  
        return ans
```

303. Range Sum Query - Immutable

```
class NumArray:  
  
    def __init__(self, nums: List[int]):  
        self.n = len(nums)  
        self.tree = [0] * 4 * self.n  
        self.build(0, 0, self.n - 1, nums)  
    def build(self, node, start, end, nums):  
        if start == end:  
            self.tree[node] = nums[start]  
            return  
        mid = (start+end) // 2  
        self.build(2 * node + 1, start, mid, nums)  
        self.build(2 * node + 2, mid + 1, end, nums)  
        self.tree[node] = self.tree[2*node + 1] + self.tree[2 * node + 2]  
    def sumRange(self, left, right):  
        return self.query(0, 0, self.n - 1, left, right)  
    def query(self, node, start, end, left, right):  
        if start > right or end < left:  
            return 0
```

```

if left <= start and end <= right:
    return self.tree[node]

mid = (start + end) // 2

leftValue = self.query(2 * node + 1, start, mid, left, right)
rightValue = self.query(2 * node + 2, mid + 1, end, left, right)

return leftValue + rightValue

```

[1991. Find the Middle Index in Array](#)

class Solution:

```

def findMiddleIndex(self, nums: List[int]) -> int:
    leftSum = 0
    rightSum = sum(nums)
    for i in range(len(nums)):
        if leftSum == rightSum - nums[i]:
            return i
        leftSum += nums[i]
        rightSum -= nums[i]
    return -1

```

[724. Find Pivot Index](#)

class Solution:

```

def pivotIndex(self, nums: List[int]) -> int:
    total = sum(nums)
    left = 0
    for i in range(len(nums)):
        right = total - left - nums[i]
        if left == right:
            return i
        left += nums[i]
    return -1

```

[560. Subarray Sum Equals K](#)

class Solution:

```

def subarraySum(self, nums: List[int], k: int) -> int:

```

```

count = 0
prefix = 0
map = {0: 1}
for num in nums:
    prefix += num
    count += map.get(prefix - k, 0)
    map[prefix] = map.get(prefix, 0) + 1
return count

```

525. Contiguous Array

class Solution:

```

def findMaxLength(self, nums):
    mp = {0: -1}
    prefix = 0
    ans = 0
    for i in range(len(nums)):
        if nums[i] == 0:
            prefix -= 1
        else:
            prefix += 1
        if prefix in mp:
            ans = max(ans, i - mp[prefix])
        else:
            mp[prefix] = i
    return ans

```

704. Binary Search

class Solution:

```

def search(self, nums: List[int], target: int) -> int:
    l, r = 0, len(nums) - 1
    while l <= r:
        mid = (l + r) // 2
        if nums[mid] == target:

```

```

        return mid
    if nums[mid] > target:
        r = mid-1
    else:
        l = mid+1
    return -1

```

35. Search Insert Position

class Solution:

```

    def searchInsert(self, nums: List[int], target: int) -> int:
        low, high = 0, len(nums)
        while low < high:
            mid = (low + high)//2
            if nums[mid] == target:
                return mid
            elif nums[mid] < target :
                low = mid + 1
            else:
                high = mid
        return low

```

69. Sqrt(x)

class Solution:

```

    def mySqrt(self, x: int) -> int:
        x = x**0.5
        return int(x)

```

34. Find First and Last Position of Element in Sorted Array

class Solution(object):

```

    def searchRange(self, nums, target):
        lst = [-1,-1]
        lst[0] = self.first_position(nums, target)
        lst[1] = self.second_position(nums, target)
        return lst

```

```

def first_position(self,nums, target):
    start = 0
    end = len(nums)-1
    ans = -1
    while start <= end:
        mid = (start+end)//2
        if target > nums[mid]:
            start = mid + 1
        elif target < nums[mid]:
            end = mid - 1
        else:
            ans = mid
            end = mid-1
    return ans

def second_position(self,nums, target):
    start = 0
    end = len(nums)-1
    ans = -1
    while start <= end:
        mid = (start+end)//2
        if target > nums[mid]:
            start = mid+1
        elif target < nums[mid]:
            end = mid-1
        else:
            ans = mid
            start = mid+1
    return ans

```

[33. Search in Rotated Sorted Array](#)

class Solution:

```

def search(self, nums: List[int], target: int) -> int:

```

```

left, right = 0, len(nums)-1
while left <= right:
    mid = (left+right)//2
    if nums[mid] == target:
        return mid

    if nums[left] <= nums[mid]:
        if nums[left] <= target < nums[mid]:

            right = mid - 1
        else:
            left = left + 1
    else:
        if nums[mid] < target <= nums[right]:
            left = mid+1
        else:
            right = mid - 1
return -1

```

4. Median of Two Sorted Arrays

class Solution:

```
def findMedianSortedArrays(self, nums1: List[int], nums2: List[int]) -> float:
```

```
    A, B = nums1, nums2
```

```
    total = len(A) + len(B)
```

```
    half = total // 2
```

```
    if len(B) < len(A):
```

```
        A, B = B, A
```

```
    l, r = 0, len(A) - 1
```

```
    while True:
```

```
        i = (l + r) // 2
```

j = half - i - 2

Aleft = A[i] if i >= 0 else float("-infinity")

Aright = A[i + 1] if (i + 1) < len(A) else float("infinity")

Bleft = B[j] if j >= 0 else float("-infinity")

Bright = B[j + 1] if (j + 1) < len(B) else float("infinity")

if Aleft <= Bright and Bleft <= Aright:

if total % 2:

return min(Aright, Bright)

return (max(Aleft, Bleft) + min(Aright, Bright)) / 2

elif Aleft > Bright:

r = i - 1

else:

l = i + 1